

Relatório de Evolução do Projeto

ChargeGrid AI Assistant

EV Challenge 2026 · FIAP + GoodWe Brasil — Sprint 03
Prompt and Artificial Intelligence · 2º Semestre CC · 2026.2
Professor Jorge Luiz Gomes

1. Resumo da evolução

Nas Sprints 1 e 2 o ChargeGrid AI Assistant era um chatbot manual: chamadas diretas à API da Groq (`chatbot.py`), um `system_prompt` gerado por f-string, histórico de conversa em uma lista Python crescendo sem limite de tokens, saída sempre em texto livre e uma camada de RAG (`rag.py`) com embeddings via `sentence-transformers` rodando localmente.

Na Sprint 03 o núcleo conversacional foi refeito em LangChain LCEL (`ChatPromptTemplate` | `ChatOllama` | `parser`), conforme o Módulo 1 do curso (Aulas 01–04). O modelo passou a rodar exclusivamente no **Ollama Cloud** (`gpt-oss:120b`), eliminando qualquer dependência de API paga (Groq) ou de modelo rodando localmente. A memória de conversa passou a ser por sessão e limitada por tokens (`RunnableWithMessageHistory`), a saída pode ser estruturada e validada com Pydantic v2 (`ConsultaRecarga`) e o `system prompt` foi reescrito com XML tagging e versionado em `prompts/`. Guardrails de escopo e anti-jailbreak, antes implícitos apenas no prompt, agora também existem como código testável em `src/guardrails/`. O módulo de RAG da Sprint 02 foi mantido no repositório apenas como referência histórica: RAG está fora do escopo exigido nesta sprint (ver PDF do Challenge, seção 3) e o modelo de embeddings que ele usa roda localmente, o que conflita com a exigência desta sprint de só usar modelos de IA via Ollama Cloud — por isso ele não é chamado pelo novo núcleo.

2. Refatoração — decisões técnicas e trade-offs

2.1 Groq → Ollama Cloud

A Sprint 02 usava uma lista de modelos Groq com fallback automático (`MODELOS` em `chatbot.py`). Essa estratégia de múltiplos modelos foi removida — não porque fallback seja ruim, mas porque o requisito desta sprint é usar somente Ollama Cloud e não é exigida chamada multi-provider (isso é o item de bônus, deixado de fora desta entrega por decisão explícita do grupo). A troca de provedor foi isolada em `src/chain/builder.py::build_llm()`: trocar de modelo é um parâmetro, não uma reescrita de código.

2.2 Memória: `RunnableWithMessageHistory` em vez de `ConversationChain`

A Aula 02 ensina `ConversationBufferMemory`, `ConversationSummaryMemory` e `ConversationTokenBufferMemory` via `ConversationChain` (pacote `langchain_classic`). Essas classes já saem com `LangChainDeprecationWarning` na própria aula, recomendando migrar para `RunnableWithMessageHistory` ou `create_agent`. Optamos por `RunnableWithMessageHistory` por ser a peça que se encaixa diretamente na chain LCEL da Aula 01, sem reescrever o núcleo conversacional em torno de um agente. Ver também o problema nº 1 na seção 4.

2.3 RAG fora do escopo desta sprint

O PDF do Challenge (seção 3) lista explicitamente RAG como algo a **não exigir** nesta sprint — fica para o CKP02/Módulo 2. Como o `rag.py` da Sprint 02 depende de um modelo de embeddings local (`sentence-transformers`), mantê-lo ativo no novo núcleo contradiria a exigência desta sprint de usar somente modelos via Ollama Cloud. Decisão: preservar o arquivo no repositório como histórico, sem importá-lo em nenhum lugar do código da Sprint 03.

3. Comparativo antes/depois (evidência do refactor)

Tabela obrigatória (seção 8.3 do PDF do Challenge). Todas as células abaixo foram preenchidas automaticamente a partir de `evals/sprint3_results.json` e `prompts/tokens_resultado.json` — nenhum número foi inventado para este relatório.

Critério	Sprints 1/2 (versão manual/legado)	Sprint 03 (LCEL)
Provedor / modelo	Groq API — llama-3.3-70b-versatile + 4 modelos de fallback	Ollama Cloud — gpt-oss:120b (configurável, sem custo)
Núcleo conversacional	Função Python manual: monta mensagens, chama a API, concatena histórico numa lista	Chain LCEL: ChatPromptTemplate ChatOllama StrOutputParser
Memória	Lista Python (historico) sem limite — cresce a cada turno	RunnableWithMessageHistory + janela deslizante por tokens (TokenBufferChatMessageHistory)
Saída estruturada	Não existe — sempre texto livre	PydanticOutputParser + schema ConsultaRecarga (com field_validator)
System prompt	f-string única, sem versionamento	XML tagging, versionado em prompts/ (v1 → v2)
Guardrails	Só o que o texto do prompt conseguia garantir	Código dedicado em src/guardrails/ (escopo, risco profissional, jailbreak) + prompt
Qualidade das respostas (nota no eval)	6/7 "Adequada" no resultados_testes.md original	5/6 "Adequada" no happy path refeito (evals/sprint3_results.json)
Tokens por turno	Prompt em bloco único (ver prompts/system_prompt_v1.md)	v1: 531 tok · v2: 486 tok (redução de 8.5%, prompts/tokens_resultado.json)
Latência média	0,8 s (resultados_testes.md, Groq)	1.50 s (média de 9 chamadas, Ollama Cloud)
Acurácia do structured output	Não se aplica (não existia)	3/3 casos corretos (100.0%) — extração + validação de schema, evals/sprint3_results.json

4. Problemas encontrados e soluções

Problema 1 — ConversationTokenBufferMemory não é compatível com RunnableWithMessageHistory

O item 2 do escopo da sprint pede explicitamente "RunnableWithMessageHistory + ConversationTokenBufferMemory". Na prática, essas duas peças não se encaixam:

RunnableWithMessageHistory exige que `get_session_history()` devolva um objeto `BaseChatMessageHistory` (com `.messages` e `add_messages()`), enquanto `ConversationTokenBufferMemory` (pacote `langchain_classic`, já deprecado) implementa outra interface (`load_memory_variables` / `save_context`). Ligar as duas peças diretamente quebra em tempo de execução.

Decisão: implementar `TokenBufferChatMessageHistory` (`src/chain/memoria.py`), uma classe própria que herda de `InMemoryChatMessageHistory` (compatível com `RunnableWithMessageHistory`) e reproduz o comportamento de janela deslizante por tokens da `ConversationTokenBufferMemory`, contando tokens com `tiktoken` — mesma biblioteca já usada na Aula 04. O resultado cumpre o objetivo pedido (memória por sessão, com limite de tokens, no padrão moderno do LangChain) sem depender de uma classe deprecada e incompatível.

Problema 2 — whitelist de palavras do domínio gerava falso positivo

A primeira versão do guardrail de escopo usava uma lista de palavras permitidas ("eletroposto", "carro elétrico", "veículo elétrico" etc.). Ao reexecutar o eval herdado da Sprint 02, o Teste 07 ("Qual o melhor carro elétrico para comprar em 2026?" — que deve ser **recusado** por estar fora do escopo) passava pelo filtro porque continha a expressão "carro elétrico", tratada erradamente como sinal de domínio.

Decisão: inverter a estratégia para uma blocklist (`src/guardrails/scope_validator.py`): só bloqueia automaticamente padrões claramente fora do domínio (compra de carro, previsão do tempo, receitas, etc.); perguntas ambíguas seguem para o LLM, que tem a mesma instrução de escopo reforçada no <escopo> do system prompt v2 — defesa em duas camadas (código + prompt) em vez de uma única regra frágil.

Problema 3 — reprodutibilidade do eval sem custo

Como o projeto não pode depender de API paga, o eval set (`evals/eval_set.py`) e o script `evals/run_evals.py` foram desenhados para rodar inteiramente contra o Ollama Cloud gratuito.

Decisão: qualquer integrante do grupo com uma conta gratuita em `ollama.com` consegue reproduzir os mesmos 13 casos de teste e regravar `evals/sprint3_results.json`, sem depender de créditos de terceiros.

Problema 4 — JSON truncado na saída estruturada derrubava o eval inteiro

Na primeira execução real de `evals/run_evals.py` (após ligar a rede), a chain de saída estruturada (`build_structured_chain`) falhou com `OutputParserException: Invalid json output`.

Causa: o `num_predict` padrão (512) era suficiente para respostas de chat, mas não para o JSON de `ConsultaRecarga` (com lista de estações aninhadas) somado ao raciocínio interno de um modelo "reasoning" como o `gpt-oss:120b` — a saída saía cortada no meio do JSON. Além disso, o script só previa `ValidationError` (schema errado), não `OutputParserException` (JSON inválido), então uma única falha de parsing derrubava o processo inteiro e descartava os resultados já coletados de happy path, memória e guardrails.

Decisão: aumentar `num_predict` para 1024 só na chain estruturada, reforçar no prompt para responder somente o JSON final (sem raciocínio visível), e fazer `_rodar_structured_output()` tentar de novo uma vez antes de marcar o caso como reprovado — sem nunca deixar uma falha de parsing interromper o restante do eval.

5. Equipe e divisão de trabalho

Nome	RM	Tarefa principal na Sprint 03
Miguel Marcelo Alves Ramos de Oliveira	569467	Elaboração e debug dos códigos
Felipe de Oliveira Doern	568798	Realização de testes
Tom Stringasci Albuquerque Coelho de Moraes	568844	Criação dos system prompts
Eric dos Santos Mendes da Silva	569528	Criação do relatório
Ligia de Andrade Matheus	568973	Verificação de corretude do projeto